Natural Language Processing

Prof. Ahmed Guessoum
The National Higher School of AI

Chapter 14

Question Answering, Information Retrieval, and Retrieval-Augmented Generation

Question Answering Systems (QAS)

- QASs are designed to fill human information needs.
- Since a lot of information is in text form (web, books, emails, etc.), QA is closely related to the task behind search engines.
- QASs often focus on factoid questions (What, Where, When, etc.) that can be answered with simple facts expressed in short or medium-length texts. E.g.:
  - `What time does the Bardo Museum open?`
  - `Where does the energy in a nuclear explosion come from?`
  - `When did Algeria get its independence?`

QASs using LLMs

- Modern NLP systems answer these questions using LLMs, in one of two ways.
  - The first is to prompt a pretrained LLM that has been finetuned on question/answer datasets with the question in the prompt.

  E.g. causal language model with a string like:

  `Q: Where is the Louvre Museum located? A:`

  have it do conditional generation given this prefix, and take the response as the answer.
  - Idea: LMs have read a lot of facts in their pretraining data, (presumably including the location of the Louvre), and have encoded this information in their parameters.

Problems with prompting an LLM

- Simply prompting an LLM can be a useful approach to answer many factoid questions. But it is not yet a complete solution for QA.
- Problems:
  - LLMs often **hallucinate**.
  - When asked questions, LLMs can make up answers that sound reasonable.
  - E.g.: Dahl et al. (2024) found that when asked questions about the legal domain (e.g. particular legal cases), LLMs hallucinated 69% to 88% of the time!
- Not always possible to tell when LMs are hallucinating, partly because LLMs aren't well-calibrated.
- In a **calibrated system**, the confidence of a system in the correctness of its answer is highly correlated with the probability of an answer being correct ➔ if a calibrated system is wrong, at least it might hedge its answer or tell us to go check another source.

Problems with prompting an LLM

- Other problems:
  - Simply prompting a LLM doesn't allow one to ask questions about proprietary data (e.g. personal emails, medical records, internal company documents, …).
  - Static LLMs have problems with questions about rapidly changing information (e.g. about an event that happened last week) since LLMs won't have up-to-date information from after their release data.
- For this reason the most common way to do question-answering with LLMs is retrieval-augmented generation or RAG.
- In RAG information retrieval (IR) techniques are used to retrieve documents likely to have information that might help answer the question. Then a LLM is used to generate an answer given these documents.

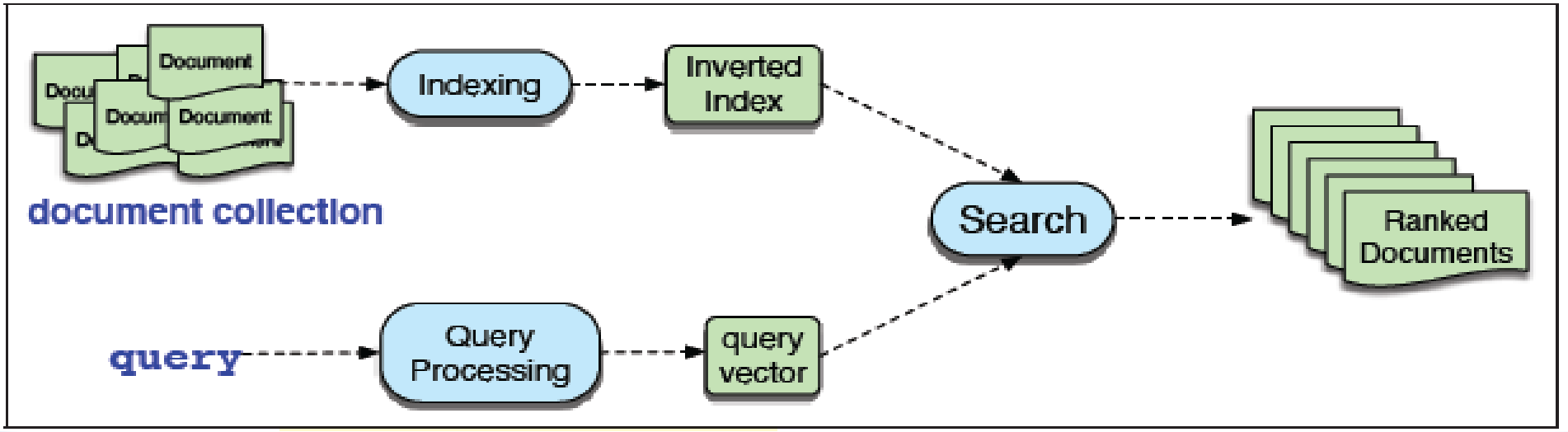
RAG for the LLM IR problems

- Basing the answers on retrieved documents can solve some of the problems with using simple prompting to answer questions.
  - It helps ensure that the answer is grounded in facts from some curated dataset.
  - The system can give the user the answer accompanied by the context of the passage or document the answer came from ➔ This information can help users have confidence in the accuracy of the answer (or help them spot when it is wrong!).
  - These retrieval techniques can be used on any proprietary data we want, such as legal or medical data for those applications.

Information Retrieval (IR)

- **Information Retrieval**: the field encompassing the retrieval of all forms of media based on user information needs.
- The resulting IR system is often called a **search engine**.
- We consider **ad hoc retrieval** where a user poses a query to a retrieval system, which then returns an ordered set of documents from some collection.
- A **document**: refers to any unit of text the system indexes and retrieves (web pages, scientific papers, news articles, or even shorter passages like collection paragraphs).
- A **collection:** refers to a set of documents being used to satisfy user term requests.
- A **term**: refers to a word in a collection, but it may also include phrases.
- A **query:** a user's information need expressed as a set of terms.

High-level architecture of an ad-hoc retrieval engine



- The basic IR architecture uses the vector space model.
- Queries and documents are mapped to vectors based on unigram word counts.
- Cosine similarity between the vectors used to rank potential documents.
- This is an example of the bag-of-words model (ref. Chap. 4), since words are considered independently of their positions.

Term weighting and document scoring

- In IR raw word counts are not used.
- Instead, a term weight is computed for each document word.
- Two term weighting schemes are common:
  - The tf-idf weighting seen previously, and
  - BM25, a slightly more powerful variant.

Reminder: TF-IDF

- **Term Frequency:** Usually the log_{10} of the word frequency is used, rather than the raw count.
  - Intuition: a word appearing 100 times in a document doesn't make that word 100 times more likely to be relevant to the meaning of the document.

$$tf_{t,d} = \begin{cases} 1 + log_{10}\, count(t,d) & if\ count(t,d) > 0 \\ 0 & otherwise \end{cases}$$

- **Document frequency:** df_t of a term t is the number of documents it occurs in.
  - Terms that occur in only a few documents are useful for discriminating those documents from the rest of the collection; terms that occur across the entire collection aren't as helpful.

$$IDF_t = log_{10} \frac{N}{df_t}$$

  - **TF-IDF value** for word t in document d:

$$tf\text{-}idf(t,d) = tf_{t,d}\ x\ IDF_t$$

Document Scoring

- Given a query q, a document d is scored by the cosine of its vector d with the query vector q:

$$score(q,d) = \cos(q,d) = \frac{q.d}{|q||d|}$$

- This can be seen as dot product of unit vectors:

$$score(q,d) = \cos(q,d) = \frac{q}{|q|} \cdot \frac{d}{|d|}$$

- This can be spelled out as:

$$\text{score}(q,d) = \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

Example

- Let us assume all words in the following query and documents are downcased and punctuation is removed:

Query: sweet love
Doc 1: Sweet sweet nurse! Love?
Doc 2: Sweet sorrow
Doc 3: How sweet is love?
Doc 4: Nurse!

$$\text{score}(q,d) = \sum_{t \in q} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

				Query		
word	cnt	tf	df	idf	tf-idf	n'lized = tf-idf/$\|q\|$
sweet	1	1	3	0.125	0.125	0.383
nurse	0	0	2	0.301	0	0
love	1	1	2	0.301	0.301	0.924
how	0	0	1	0.602	0	0
sorrow	0	0	1	0.602	0	0
is	0	0	1	0.602	0	0
$\|q\| = \sqrt{.125^2 + .301^2} = .326$						

Computation of the tf-idf cosine between query and Doc 1, and query and Doc 2.

	Document 1					Document 2				
word	cnt	tf	tf-idf	n'lized	$\times$ q	cnt	tf	tf-idf	n'lized	$\times$q
sweet	2	1.301	0.163	0.357	**0.137**	1	1.000	0.125	0.203	**0.0779**
nurse	1	1.000	0.301	0.661	0	0	0	0	0	0
love	1	1.000	0.301	0.661	**0.610**	0	0	0	0	0
how	0	0	0	0	0	0	0	0	0	0
sorrow	0	0	0	0	0	1	1.000	0.602	0.979	0
is	0	0	0	0	0	0	0	0	0	0
$\|d_1\| = \sqrt{.163^2 + .301^2 + .301^2} = .456$						$\|d_2\| = \sqrt{.125^2 + .602^2} = .615$				
Cosine: $\sum$ of column:					**0.747**	Cosine: $\sum$ of column:				**0.0779**

Variants of Tf-Idf

- In practice, there are many variants and approximations to the previous equation.
- E.g. dropping the idf term for the document (computed twice in the equation):

$$\text{score}(q,d) = \sum_{t \in \mathbf{q}} \frac{\text{tf}_{t,q} \cdot \text{idf}_t}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf}_{t,d} \cdot \text{idf}_t}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

- This sometimes results in better performance.

Another term weighting scheme: BM25

- **BM25** (Best Matching 25) ranks documents based on their relevance to a given query.
- BM25 is an extension of the **TF-IDF** model, designed to improve performance by incorporating the idea that term frequency should have diminishing returns as the TF increases.
- While TF-IDF assumes a linear relationship between TF and relevance, BM25 introduces non-linear effects to better match real-world information retrieval needs, especially when the TF grows very high.

$$score(d,q) = \sum_{t \in q} IDF(t).\frac{tf_{t,d}.(k+1)}{tf_{t,d} + k\,.(1-b+b\,.\frac{|d|}{avgDL})}$$

- **|*d*|**: length of document ***d*** (i.e., total number of terms in d).
- ***avgDL***: average document length in the collection.
- ***k*** and ***b*** are the parameters we'll focus on.

Parameters *k* and *b* in BM25

k (Term Frequency Scaling):

- The ***k*** parameter controls the saturation effect of TF. It determines how much TF affects the document score.
- When ***k*** is low (e.g., ***k = 1***), TF has a **stronger** influence on the score because of the numerator in the formula.
- When ***k*** is high (e.g., ***k = 2*** or ***k = 3***), TF has **less impact** on the score. This means the relevance does not increase too dramatically as TF increases.
- The purpose of ***k*** is to **moderate the effect** of TF.
- For example, if a term appears 100 times in a document, ***k*** controls how much more significant that term becomes in comparison to a term that appears only once.

Parameters *k* and *b* in BM25

b (Document Length Normalization):

- The **b** parameter adjusts the effect of document length on the relevance score.
- **b** moderates the **length bias** in IR, since longer documents tend to have more terms and thus more opportunities to match query terms.
  - **b = 1** means the document length has a **strong effect** on the normalization. If a document is longer, it gets a **lower score** (because longer documents tend to be less relevant, on average).
  - **b = 1**: it fully accounts for the length of the document, giving shorter documents (with the same number of term matches) a higher score.
  - **b = 0** means there is **no length normalization**.
    - Every document is treated equally regardless of its length, which is more similar to the classic TF-IDF model.
    - BM25 essentially behaves like a basic TF-based model without adjusting for document length.

How k and b Influence BM25 vs TF-IDF

- **k** adjusts the **impact of TF**. When *k* is small, TF increases quickly with the number of occurrences. When it's large, it reduces the contribution of multiple occurrences of a term in a document.
- **b** controls the **length normalization**. A value close to 1 means longer documents are penalized more, while a value close to 0 means the length of the document doesn’t matter at all.

Together, **k** and **b** help refine BM25 by:

- **Smoothing the TF**: Avoiding situations where an excessively high TF leads to an overinflated document score.
- **Normalizing for document length**: Preventing longer documents from unfairly boosting their relevance scores just because they contain more terms.
- Manning et al. (2008) suggest reasonable values are

  k = [1.2, 2] and b = 0.75.

Practical Example:

- Consider the two documents:
  - **Doc A**: "Information retrieval is important" (3 terms).
  - **Doc B**: "Information retrieval is important for efficient data retrieval" (6 terms).
- If both documents contain the same term "information", but **Doc B** is longer, **Doc A** will likely be ranked higher, depending on the value of **b**.
  - With **b = 1**, Doc A will benefit from being shorter, as it matches the query term without the penalty of being overly long.
- If **k = 1**, the TF is more sensitive, meaning that even a small increase in frequency will have a relatively large impact.
- If **k = 2**, the effect is dampened, so adding extra occurrences of the term has a smaller effect.

Stop words

- **Intuition**: High-frequency terms (often function words like *the*, *a*, *to*):
  - carry little semantic weight,
  - may not help with retrieval, and
  - can help shrink the inverted index files (described below).
- Downside of using a stop list: it makes it difficult to search for phrases that contain words in the stop list.
  - For example, common stop lists would reduce the phrase ***to be or not to be*** to the phrase ***not***!
- In modern IR systems, the use of stop lists is much less common, partly due to improved efficiency and partly because much of their function is already handled by IDF weighting.
- Stop word removal is occasionally useful in various NLP tasks.

Inverted Index

- Basic search problem in IR: find all documents d ∈ C that contain a term q ∈ Q.
- The data structure for this task is the ***inverted index***.
- Used for making the search efficient, and conveniently storing useful information like the DF and the count of each term in each document.
- The ***inverted index*** consists of two parts: (1) a dictionary and (2) the postings.
  - Dictionary: a list of terms (designed to be efficiently accessed), each pointing to a postings list for the term.
  - Postings list: the list of document IDs associated with each term,
    - can also contain information like the TF or even the exact positions of terms in the document.

Example:

- For the 4 documents seen in slide 14:

how {1} → 3 [1]
is {1} → 3 [1]
love {2} → 1 [1] → 3 [1]
nurse {2} → 1 [1] → 4 [1]
sorry {1} → 2 [1]
sweet {3} → 1 [2] → 2 [1] → 3 [1]

- Given a list of terms in *query*, we can very efficiently get lists of all candidate documents, together with the information necessary to compute the tf-idf scores we need.
- For the question-answering domain of finding Wikipedia pages to match a *user query*, Chen et al. (2017a) show that indexing based on bigrams works better than unigrams, and use efficient hashing algorithms rather than the inverted index to make the search efficient.

Evaluation of IR Systems

- The performance of retrieval systems is measured using the same precision and recall metrics.
- **Assumption:** each document returned by the IR system is either relevant to our purposes or not relevant.
- **Precision:** fraction of the returned documents that are relevant.
- **Recall:** fraction of all relevant documents that are returned.

$$Precision = \frac{|R|}{|T|} \qquad Recall = \frac{|R|}{|U|}$$

where

- T: ranked documents returned by the system in response to an information request,
- R: subset of relevant documents, and
- U: documents in the collection as a whole relevant to the request.

Precision and Recall for IR Systems

- Precision and Recall don’t adequately measure the performance of an IR system that ranks the documents it returns.
- If we are comparing the performance of two ranked retrieval systems, we need a metric that prefers the one that ranks the relevant documents higher.
- We need to adapt Precision and Recall to capture how well a system does at putting relevant documents higher in the ranking.

Example

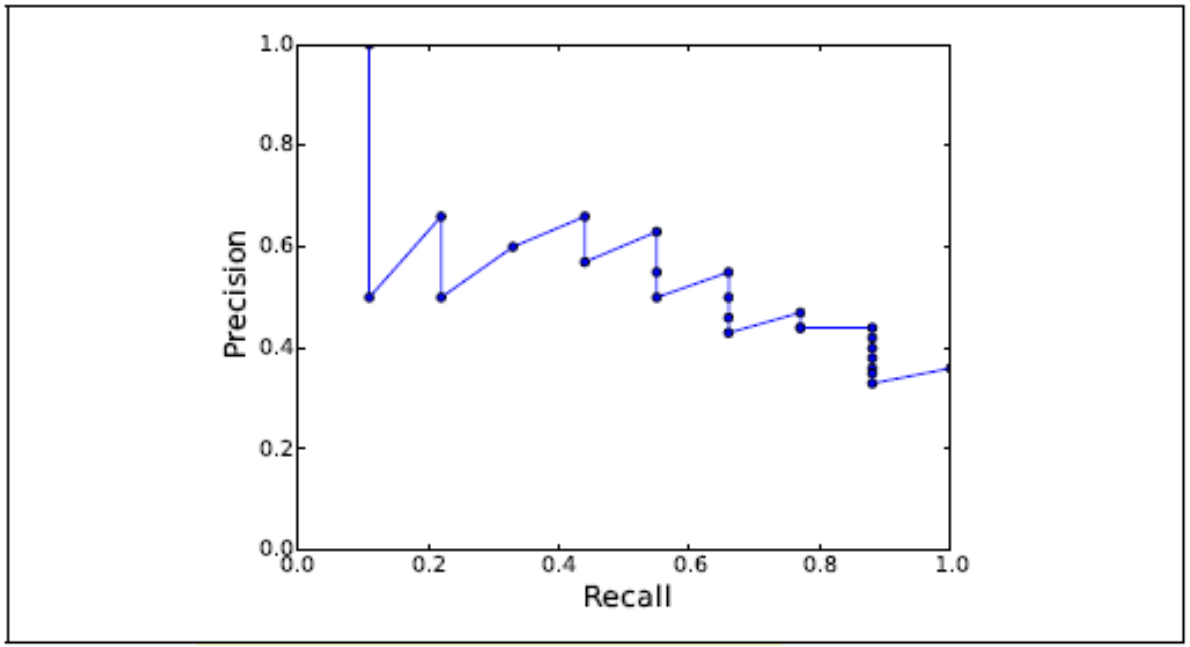
- Consider the following table which gives **rank-specific precision** and **recall** values calculated as we proceed down through a set of ranked documents for a particular query;
- There are 9 relevant documents in the collection to the query .

Rank	Judgment	Precision_{Rank}	Recall_{Rank}
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44
8	R	.63	.55
9	N	.55	.55
10	N	.50	.55
11	R	.55	.66
12	N	.50	.66
13	N	.46	.66
14	N	.43	.66
15	R	.47	.77
16	N	.44	.77
17	N	.44	.77
18	R	.44	.88
19	N	.42	.88
20	N	.40	.88
21	N	.38	.88
22	N	.36	.88
23	N	.35	.88
24	N	.33	.88
25	R	.36	1.0

Precision-Recall Curve

Notes:

- Recall is non-decreasing; when a relevant document is encountered, recall increases, and when a non-relevant document is found it remains unchanged.
- Precision, jumps up and down, increasing when relevant documents are found, and decreasing otherwise.
- The most common way to visualize precision and recall is to plot precision against recall in a **precision-recall curve**,



Interpolated Precision

- The above **precision-recall curve** shows the values for a single query. We'll need
  - to combine values for all the queries, and
  - in a way that lets us compare one system to another.
- The main challenge with Precision-Recall curve is that precision is often **not monotonic** with respect to recall
  ➔ This means that as we retrieve more items (increasing recall), precision might fluctuate (sharp dips and sharp increases).
  ➔ Difficult to compare models or systems based on such curves.
- **Interpolated precision** provides a **smoothed version** of precision that focuses on avoiding the sharp drops in precision.
- At any given recall level, we calculate the **maximum precision** achieved at **any recall level greater than or equal to the current level**.

Interpolated Precision

- One way of doing this is to plot averaged precision values at 11 fixed levels of recall (0 to 100, in steps of 10).
- Since we're not likely to have data points at these exact levels, we use interpolated precision values for the 11 recall values from the data points we do have.
- This is done by choosing the maximum precision value achieved at any level of recall at or above the current level.

$$IntPrecision(r) = \max_{i \geq r} Precision(i)$$

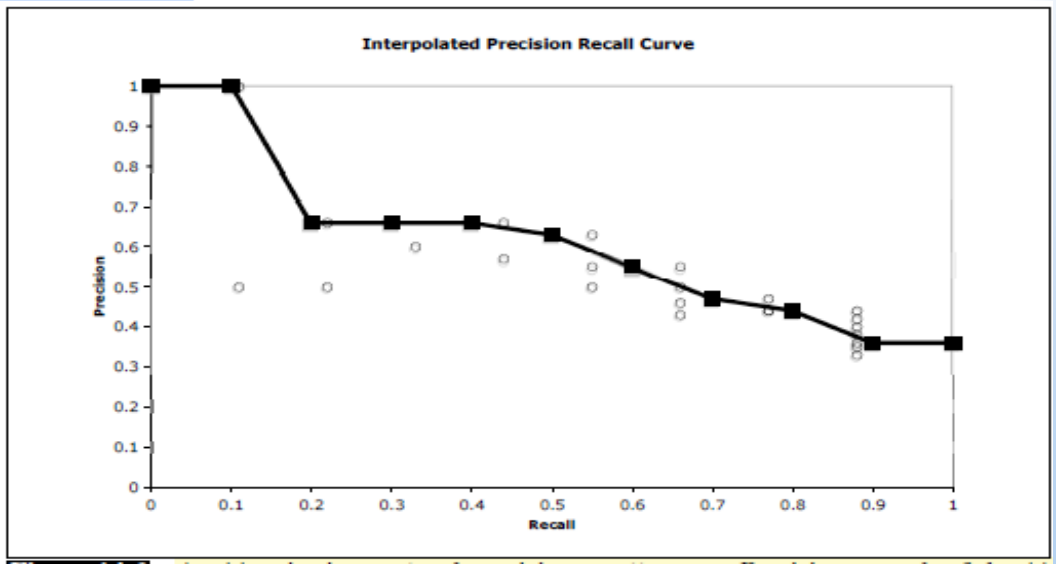
- Particularly useful for evaluating systems where precision tends to vary with the volume of retrieved items (as recall increases), and it gives a more **robust measure of precision** across different recall levels.
- Prevents penalizing a model for sharp decreases in precision.

Interpolated Precision

- This interpolation scheme lets us average performance over a set of queries, and helps smooth over the irregular precision values in the original data.

Rank	Judgment	Precision_{Rank}	Recall_{Rank}
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44
8	R	.63	.55
9	N	.55	.55
10	N	.50	.55
11	R	.55	.66
12	N	.50	.66
13	N	.46	.66
14	N	.43	.66
15	R	.47	.77
16	N	.44	.77
17	N	.44	.77
18	R	.44	.88
19	N	.42	.88
20	N	.40	.88
21	N	.38	.88
22	N	.36	.88
23	N	.35	.88
24	N	.33	.88
25	R	.36	1.0

Interpolated Precision	Recall
1.0	0.0
1.0	.10
.66	.20
.66	.30
.66	.40
.63	.50
.55	.60
.47	.70
.44	.80
.36	.90
.36	1.0



Interpolated Precision

- Given curves such as the previous one, we can compare two ranked systems or approaches by comparing their curves.
  - Curves that are higher in precision across all recall values are preferred.
  - These curves can also provide insight into the overall behavior of a system.
  - Systems that are higher in precision toward the left may favor precision over recall,
  - Systems that are more geared towards recall will be higher at higher levels of recall (to the right).

Mean Average Precision (MAP)

- Mean Average Precision (MAP) is another way to evaluate ranked retrieval.
- MAP provides a single metric that can be used to compare competing systems or approaches.
- As before, we descend through the ranked list of items, but now we note the precision only at those points where a relevant item has been encountered (for example at ranks 1, 3, 5, 6 but not 2 or 4 in previous Precision-Recall table).

Mean Average Precision (MAP)

- For a single query, we average these individual precision measurements over the returned set (up to some fixed cutoff).

$$AP = \frac{1}{|R_r|} \sum_{d \in R_r} Precision_r(d)$$

where R_r is the set of relevant documents at or above r, and

$Precision_r(d)$ is the precision measured at the rank at which document d was found.

- For an ensemble of queries Q, we average over these averages, to get the final MAP measure:

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(Q)$$

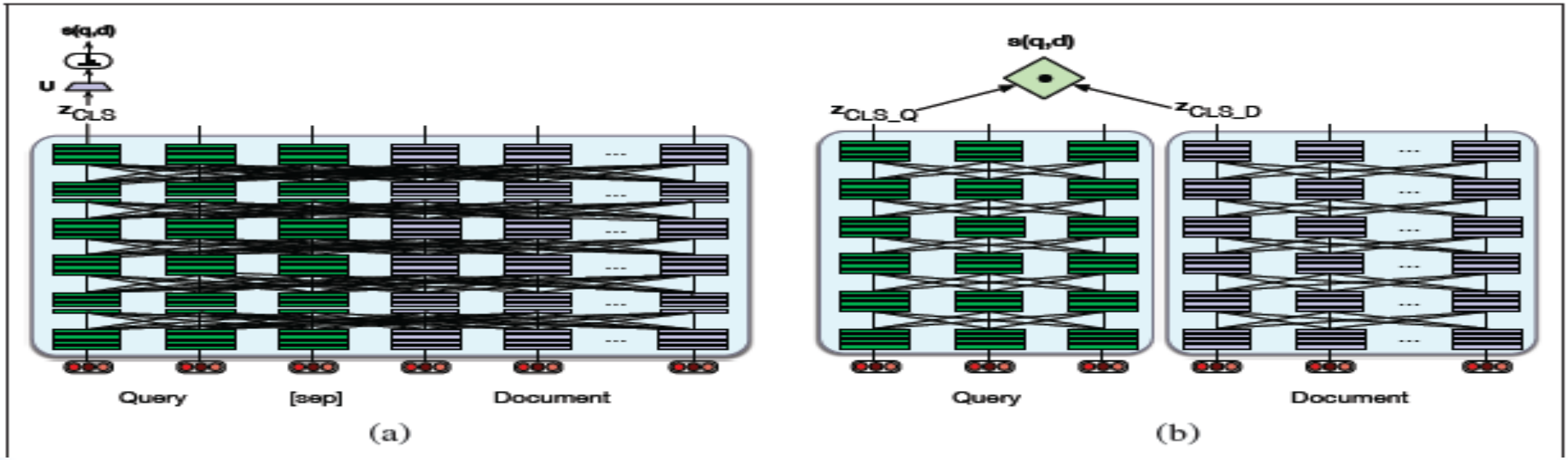
- The MAP for the single query (hence = AP) in previous Precision-Recall table is 0.6.

Information Retrieval with Dense Vectors

- Conceptual flaw with the classic tf-idf or BM25 algorithms for IR: they work only if there is exact overlap of words between the query and document.
- **Vocabulary mismatch problem** : the user posing a query (needs to guess exactly what words are used in the answer).
- **Solution**: use an approach that can handle synonymy instead of (sparse) word-count vectors ➔ use (dense) embeddings.
- Implemented in modern times via encoders like BERT.
- **Most powerful approach**:
  - present both *q* and *d* to a single encoder,
  - the transformer self-attention will see all the tokens of both *q* and *d*,
  - It builds a representation that is sensitive to the meanings of both *q* and *d*.
  - Then a linear layer can be put on top of the [CLS] token to predict a similarity score for the query/document tuple:

$$z = \text{BERT}(q;[\text{SEP}];d)[\text{CLS}]$$
$$\text{score}(q,d) = \text{softmax}(\mathbf{U(z)})$$

Dense retrieval architecture



- Two ways to do dense retrieval, illustrated by using lines between layers to schematically represent self-attention:
  (a) Use a single encoder to jointly encode *q* and *d* and finetune to produce a relevance score with a linear layer over the CLS token. This is too compute-expensive to use except in rescoring.
  (b) Use separate encoders for *q* and *d* , and use as the score the dot product between CLS token outputs for the *q* and *d*. This is less compute-expensive, but not as accurate.
- The model can create a meaningful, dense vector (CLS) that summarizes the semantic content (i.e. similarity or not) of the entire text (query and/or document).
- This vector can be used directly for ranking and retrieval.

Note on Rescoring

- **Rescoring** refers to a step that happens **after an initial retrieval** process.
- In a typical retrieval system, one might
  - first use a less computationally expensive method (such as dot-product or separate encoders) to retrieve a set of candidate documents that are likely to be relevant.
  - Then, one might use the more expensive method (joint encoding with a single encoder) to "rescore" these candidates by re-evaluating their relevance in more detail.
- Since the joint encoding with a single encoder is too expensive to apply to all documents in a large dataset, it is more feasible to use it only on a smaller set of top candidates that were retrieved quickly using a cheaper method.

Retrieval Optimisation

- Usually the retrieval step is not done on an entire document.
- Instead,
  - documents are broken up into smaller passages, such as non-overlapping fixed-length chunks of say 100 tokens, and
  - the retriever encodes and retrieves these passages rather than entire documents.
- The *q* and *d* have to be made to fit in the BERT 512-token window, for example by truncating the *q* to 64 tokens and truncating the *d* if necessary so that it, the query, [CLS], and [SEP] fit in 512 tokens.
- The BERT system together with the linear layer U can then be fine-tuned for the relevance task by gathering a finetuning dataset of relevant and non-relevant passages.

Dealing with computation constraints

- Problem with full BERT architecture in **Fig. a** is the expense in computation and time.
  - With this architecture, for each query, one has to pass every single document in the entire collection through a BERT encoder jointly with the new query!
  - ➔ This enormous use of resources is impractical for real cases.
- A much more efficient architecture is **the bi-encoder** (**Fig. b**).
  - The documents in the collection are encoded only one time using two separate encoder models, one to encode the query and one to encode the document.
  - All the encoded document vectors are stored in advance.
  - When a query comes in,
    - just this query is encoded and
    - then use the dot product between the query vector and the precomputed document vectors as the score for each candidate document.

Dealing with computation constraints

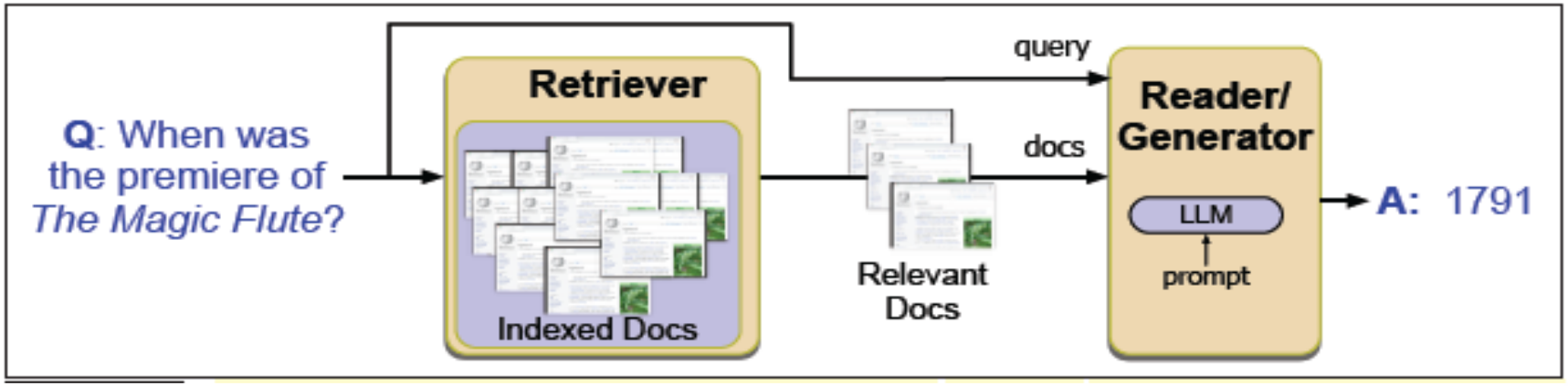
- For example, if we used BERT, we would have two encoders BERT_Q and BERT_D and we could represent the q and d as the [CLS] token of the respective encoders:

$$z_q = \text{BERT}_Q(q)\ [\text{CLS}]$$
$$z_d = \text{BERT}_D(d)\ [\text{CLS}]$$
$$\text{score}(q, d) = z_q \cdot z_d$$

- The bi-encoder is
  - much cheaper than a full query-document encoder, but
  - is less accurate, since its relevance decision can't take full advantage of all the possible meaning interactions between all the tokens in q and the tokens in d.
- There are numerous intermediate approaches that lie in between the full encoder and the bi-encoder.
  - One alternative: use cheaper methods (like BM25) as the first pass relevance ranking for each document, take the top N ranked documents, and use expensive methods like the full BERT scoring to re-rank only the top N documents rather than the whole set.

Answering Questions with RAG

- Dominant paradigm for question answering: answer a user's question by
  - First finding supportive text segments from the web or another large collection of documents, and
  - then generating an answer based on the documents.
- **Retrieval-Augmented Generation (RAG)**: method of generating based on retrieved documents.
- The two components are sometimes called the **retriever** and the **reader**.



- In RAG, we take a large pretrained language model, give it the set of retrieved passages and other text as its prompt, and autoregressively generate a new answer token by token.

Retrieval-Augmented Generation

- Recall that in simple conditional generation, we can cast the task of question answering as word prediction by giving a language model a question and a token like `A:` suggesting that an answer should come next:

  `Q: Who wrote the book “The Origin of Species"? A:`

- Then we generate autoregressively conditioned on this text.
- More formally, recall that simple autoregressive **language modeling** computes the probability of a string from the previous tokens: $p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i | x_{<i})$
- Simple conditional generation for **question answering** adds a prompt like *Q:* , followed by a query q , and *A:*, all concatenated: $p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p([Q:]\ ;\ q\ ;\ [A:]\ ;\ x_{<i})$

Need for RAG

- Advantage of using a LLM: enormous amount of knowledge encoded in its parameters from the text it was pretrained on.
- This kind of simple prompted generation can work fine for many simple factoid questions,
- BUT it is not a general solution for QA, because
  - it leads to hallucination,
  - is unable to show users textual evidence to support the answer, and
  - is unable to answer questions from proprietary data.
- **Idea of RAG**: address these problems by conditioning on the retrieved passages as part of the prefix, perhaps with some prompt text like "`Based on these texts, answer this question:`".

RAG

Schematic of a RAG Prompt

```
retrieved passage 1

retrieved passage 2

...

retrieved passage n

Based on these texts, answer this question:  Q: Who wrote
the book ‘‘The Origin of Species"?  A:
```

- More formally,

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid R(Q);\ prompt\ ;[Q:]\ ;\ q\ ;\ \ [A:]\ ;\ x_{<i})$$

- Successfully applying the RAG algorithm for QA requires a successful retriever, and often a two-stage retrieval algorithm is used in which the retrieval is reranked.
- Some complex questions may require **multi-hop architectures**, in which a query is used to retrieve documents, which are then appended to the original query for a second stage of retrieval.

Evaluating Question Answering

- 3 techniques are commonly employed to evaluate QA systems, with the choice depending on the type of question and QA situation.
- For multiple choice questions, **exact match** is reported, i.e. the % of predicted answers that match the gold answer exactly.
- Questions with free text answers, like Natural Questions, are commonly evaluated with token F1 score to roughly measure the partial string overlap between the answer and the reference answer:
  - **F1 score**: The average token overlap between predicted and gold answers. Treat the prediction and gold as a bag of tokens, and compute F1 for each question, then return the average F1 over all questions.

Example questions from MMLU dataset

MMLU examples

College Computer Science
Any set of Boolean operators that is sufficient to represent all Boolean expressions is said to be complete. Which of the following is NOT complete?
(A) AND, NOT
(B) NOT, OR
(C) AND, OR
(D) NAND

College Physics
The primary source of the Sun's energy is a series of thermonuclear reactions in which the energy produced is c^2 times the mass difference between
(A) two hydrogen atoms and one helium atom
(B) four hydrogen atoms and one helium atom
(C) six hydrogen atoms and two helium atoms
(D) three helium atoms and one carbon atom

International Law
Which of the following is a treaty-based human rights mechanism?
(A) The UN Human Rights Committee
(B) The UN Human Rights Council
(C) The UN Universal Periodic Review
(D) The UN special mandates

Prehistory
Unlike most other early civilizations, Minoan culture shows little evidence of
(A) trade.
(B) warfare.
(C) the development of a common religion.
(D) conspicuous consumption by elites.

Mean Reciprocal Rank (MRR)

- In some situations QA systems give multiple ranked answers. In such cases **Mean Reciprocal Rank** (**MRR**) is used.
- MRR is designed for systems that return a short ranked list of answers or passages for each test set question, which we can compare against the (human-labeled) correct answer.
- Each test set question is scored with the **reciprocal of the rank** of the first correct answer.
  - E.g.: if the system returned 5 answers to a question but the first 3 are wrong (so the highest-ranked correct answer is ranked 4th), the reciprocal rank for that question is ¼.

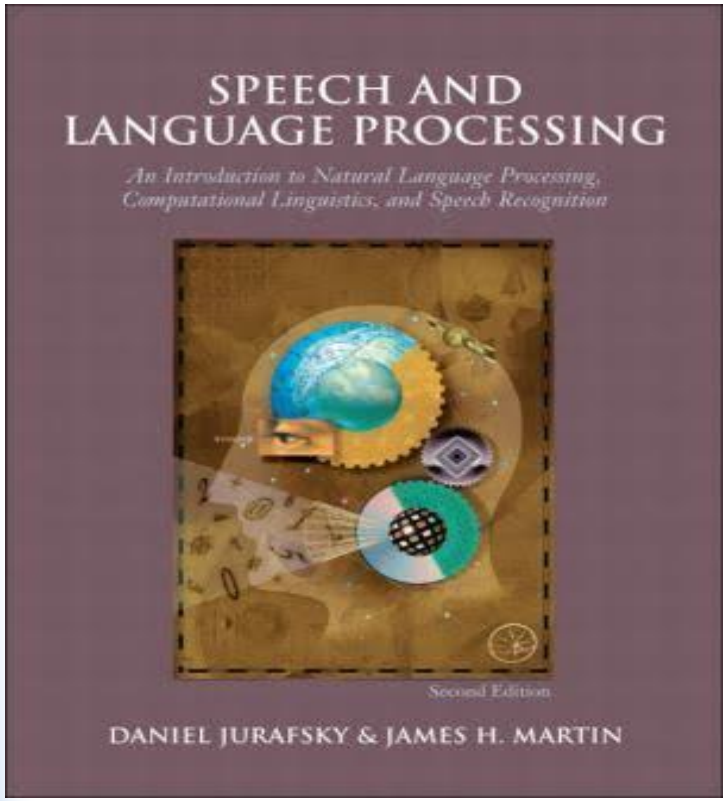
Mean Reciprocal Rank (MRR)

- *The **MRR of a system** is the average of the scores for each question in the test set.*
- In some versions of MRR, questions with a score of zero are ignored in this calculation.
- More formally, for a system returning ranked answers to each question in a test set Q, (or in the alternate version, let Q be the subset of test set questions that have non-zero scores). MRR is then defined as

$$MRR = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{1}{rank_i}$$

Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf



The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.